

SIT720 - Task 8.1D: Machine Learning Mini Project

Sydney Housing Price Prediction and Decision Support System

Name: Jason Hu

Student ID: 219220123

Project files

Project repository: <https://github.com/Joifn/8.1D-Machine-Learning-Mini-Project>

The archive holds the collected dataset, the raw copied listing pages behind it, the parser (parse_domain.py), the notebook (8.1D.ipynb), the Streamlit application and a README with build and run instructions.

Part 1 - Problem definition and data collection

The problem. Predict a property's sale price from its suburb, characteristics and sale details - supervised regression, built as a decision support tool.

Why these three suburbs. Bondi is a scarcity play: beachside land is fixed in supply, so the premium holds through downturns and you buy the postcode more than the dwelling. Chatswood is an access play - Metro, North Shore line, school catchments and a real town centre give CBD-grade access at apartment-grade entry prices. Penrith is an infrastructure play: the Western Sydney Airport and Aerotropolis spend is the only genuine re-rating story of the three, and at under \$1M for 550-650 sqm the land is cheap enough that the dwelling barely matters.

Collection. Domain has no data export, so fourteen sold-listing pages were copied as text and parsed by purpose-written code, with the raw pages kept for auditability. The result is 201 properties - 75 Bondi, 51 Chatswood, 75 Penrith - sold between 10 February and 23 September 2026.

Data quality. 73 of the 275 listings seen (27%) were Price Withheld and unusable. Domain's area field is also ambiguous: 27 apartments gave 47-229 sqm, the internal area, but 16 gave 627-28,725 sqm, the block's site area - one \$325,000 apartment against 2.9 hectares. Nothing falls between, so figures at or below 300 sqm were kept as internal area and the rest discarded.

Bias and limitations. Withholding is not random: Penrith 4%, Bondi 22%, Chatswood 49%. Vendors withhold at the top of the market and on poor results, so Chatswood's missing half is unlikely to resemble the half remaining, and Part 3 shows this is where the model fails worst. The suburbs also cover unequal time windows and 142 of 201 are apartments. Most fundamentally the data records quantity, not quality: nothing captures condition, aspect or outlook, a ceiling no algorithm can lift.

```

In [2]: DATA = Path("data/sydney_properties.csv")
        props = pd.read_csv(DATA)

        props = props[~props["address"].astype(str).str.startswith("EXAMPLE")].reset_index(drop=True)

        print(f"Properties collected: {len(props)}")
        print(f"Columns: {props.shape[1]}")
        print("\nPer suburb:")
        print(props["suburb"].value_counts().reindex(SUBURBS).to_string())

        ok_total = len(props) >= 100
        ok_each = props["suburb"].value_counts().reindex(SUBURBS).min() >= 30
        print(f"\nAt least 100 properties overall: {ok_total}")
        print(f"At least 30 in every suburb: {ok_each}")

        props["sale_date"] = pd.to_datetime(props["sale_date"])
        print(f"\nSales span {props['sale_date'].min():%d %b %Y} to {props['sale_date'].max():%d %b %Y}")

        print("\nMissing values per column:")
        miss = props.isna().sum()
        print(miss[miss > 0].to_string() if miss.sum() else " none")

        print("\nSale price summary:")
        print(props["sale_price"].describe().apply(lambda v: f"{v:,.0f}").to_string())

```

Properties collected: 201
Columns: 17

Per suburb:
suburb
Bondi 75
Chatswood 51
Penrith 75

At least 100 properties overall: True
At least 30 in every suburb: True

Sales span 10 Feb 2026 to 23 Sep 2026

Missing values per column:
land_size_sqm 122
internal_area_sqm 201
days_on_market 201
distance_to_station_km 201
agent_description 201
listing_url 201
agent_agency 54

Sale price summary:
count 201
mean 1,611,045
std 1,301,218
min 325,000
25% 675,000
50% 1,200,000
75% 2,110,000
max 8,725,000

```
In [3]: print("Data types and how complete each column is:\n")
summary = pd.DataFrame({
    "dtype": props.dtypes.astype(str),
    "non_null": props.notna().sum(),
    "missing_%": (props.isna().mean() * 100).round(1),
    "unique": props.nunique()})
print(summary.to_string())

print("\nProperty types collected in each suburb:")
print(pd.crosstab(props["suburb"], props["property_type"]).to_string())

print("\nSale price by suburb:")
print(props.groupby("suburb")["sale_price"].agg(
    n="count", median="median", mean="mean", min="min", max="max").round(0).to_string())
```

Data types and how complete each column is:

	dtype	non_null	missing_%	unique
property_id	int64	201	0.0	201
suburb	str	201	0.0	3
address	str	201	0.0	201
property_type	str	201	0.0	5
bedrooms	int64	201	0.0	7
bathrooms	int64	201	0.0	5
car_spaces	int64	201	0.0	5
land_size_sqm	float64	79	60.7	74
internal_area_sqm	float64	0	100.0	0
sale_price	int64	201	0.0	168
sale_date	datetime64[us]	201	0.0	117
sale_method	str	201	0.0	2
days_on_market	float64	0	100.0	0
distance_to_station_km	float64	0	100.0	0
agent_description	float64	0	100.0	0
listing_url	float64	0	100.0	0
agent_agency	str	147	26.9	49

Property types collected in each suburb:

property_type	Apartment	Duplex	House	Semi	Townhouse
suburb					
Bondi	57	0	17	1	0
Chatswood	31	1	17	1	1
Penrith	54	1	16	0	4

Sale price by suburb:

	n	median	mean	min	max
suburb					
Bondi	75	1735000.0	2232847.0	560000	8725000
Chatswood	51	1717500.0	2007902.0	719000	6200000
Penrith	75	610000.0	719380.0	325000	2990000

```
In [4]: print("Cleaning the area field")
print()

is_unit = props["property_type"] == "Apartment"
stated = props["land_size_sqm"].notna()

AREA_CUTOFF = 300
internal = is_unit & stated & (props["land_size_sqm"] <= AREA_CUTOFF)
site = is_unit & stated & (props["land_size_sqm"] > AREA_CUTOFF)

kept_lo, kept_hi = props.loc[internal, "land_size_sqm"].agg(["min", "max"])
cut_lo, cut_hi = props.loc[site, "land_size_sqm"].agg(["min", "max"])

props.loc[internal, "internal_area_sqm"] = props.loc[internal, "land_size_sqm"]
props.loc[is_unit, "land_size_sqm"] = np.nan

print(f"Apartments quoting an area: {int((is_unit & stated).sum())}")
print(f"  <= {AREA_CUTOFF} sqm, read as the dwelling's internal area: {int(internal.sum())}")
print(f"    these run {kept_lo:.0f} to {kept_hi:.0f} sqm")
print(f"  > {AREA_CUTOFF} sqm, discarded as the strata block's site area: {int(site.sum())}")
print(f"    these run {cut_lo:.0f} to {cut_hi:.0f} sqm, so nothing sits between "
      f"{kept_hi:.0f} and {cut_lo:.0f}")
print(f"Land size retained for {int(props['land_size_sqm'].notna().sum())} houses and "
      f"{int(props['internal_area_sqm'].notna().sum())} attached dwellings")

print()
print("Land size by property type after cleaning:")
print(props[props["land_size_sqm"].notna()].groupby("property_type")["land_size_sqm"]
      .agg(n="size", min="min", median="median", max="max").round(0).to_string())

empty = [c for c in props.columns if props[c].isna().all()]
props = props.drop(columns=empty)
print()
print(f"Columns dropped because nothing was collected into them: {empty}")
```

Cleaning the area field

Apartments quoting an area: 43
 <= 300 sqm, read as the dwelling's internal area: 27
 these run 47 to 229 sqm
 > 300 sqm, discarded as the strata block's site area: 16
 these run 627 to 28725 sqm, so nothing sits between 229 and 627
 Land size retained for 36 houses and attached dwellings

Land size by property type after cleaning:

	n	min	median	max
property_type				
Duplex	1	614.0	614.0	614.0
House	33	212.0	518.0	1067.0
Semi	1	359.0	359.0	359.0
Townhouse	1	624.0	624.0	624.0

Columns dropped because nothing was collected into them: ['days_on_market', 'distance_to_station_km', 'agent_description', 'listing_url']

Part 2 - Data understanding and feature engineering

Distribution and the suburbs. Skewness is 2.00 raw and 0.36 logged, so every model predicts log price. I expected three separated markets and was wrong: Bondi's median is \$1,735,000 and Chatswood's \$1,717,500 - a 1% gap - against Penrith's \$610,000. The division is Penrith versus the other two, and Bondi and Chatswood differ in composition, 57 apartments to 17 houses against 31 to 17, not price.

Trend and outliers. Monthly medians swing wildly, Bondi moving from \$1,330,000 in July to \$2,750,000 in August, tracking which properties sold. Later months hold more Penrith sales, producing the -0.330 correlation between month and price: composition, not trend. Eight properties fall outside the Tukey fences but are genuine - 74 Nepean Avenue, Penrith, at \$2,990,000 sits 5.8 standard deviations above its suburb mean - and removing them would teach the model that expensive properties do not exist.

Testing my prior expectations. I nominated suburb, bedrooms and land size as the strongest influences. The first two held; land size failed at 0.008, being stated for only 36 properties, and 500 sqm means different things in Penrith and Bondi so pooled effects cancel.

```
In [5]: fig, axes = plt.subplots(1, 3, figsize=(16, 4.2))

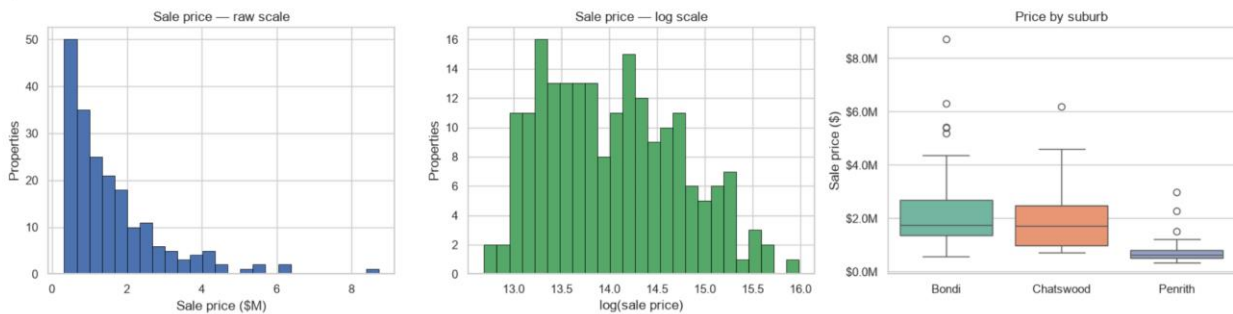
axes[0].hist(props["sale_price"] / 1e6, bins=25, color="#4c72b0", edgecolor="black", lw=0.5)
axes[0].set(xlabel="Sale price ($M)", ylabel="Properties", title="Sale price - raw scale")

axes[1].hist(np.log(props["sale_price"]), bins=25, color="#55a868", edgecolor="black", lw=0.5)
axes[1].set(xlabel="log(sale price)", ylabel="Properties", title="Sale price - log scale")

sns.boxplot(data=props, x="suburb", y="sale_price", order=SUBURBS, ax=axes[2], palette="Set2")
axes[2].set(xlabel="", ylabel="Sale price ($)", title="Price by suburb")
axes[2].yaxis.set_major_formatter(lambda v, p: f"${v/1e6:.1f}M")

plt.tight_layout(); plt.show()

skew_raw = props["sale_price"].skew()
skew_log = np.log(props["sale_price"]).skew()
print(f"Skewness on the raw scale: {skew_raw:.2f}")
print(f"Skewness after a log transform: {skew_log:.2f}")
print("\nMedian price by suburb:")
print(props.groupby("suburb")["sale_price"].median().reindex(SUBURBS).apply(lambda v: f"${v/1e6:.0f}").to_string())
```



Skewness on the raw scale: 2.00
Skewness after a log transform: 0.36

Median price by suburb:
suburb
Bondi \$1,735,000
Chatswood \$1,717,500
Penrith \$610,000

```
In [6]: fig, axes = plt.subplots(1, 3, figsize=(16, 4.2))
```

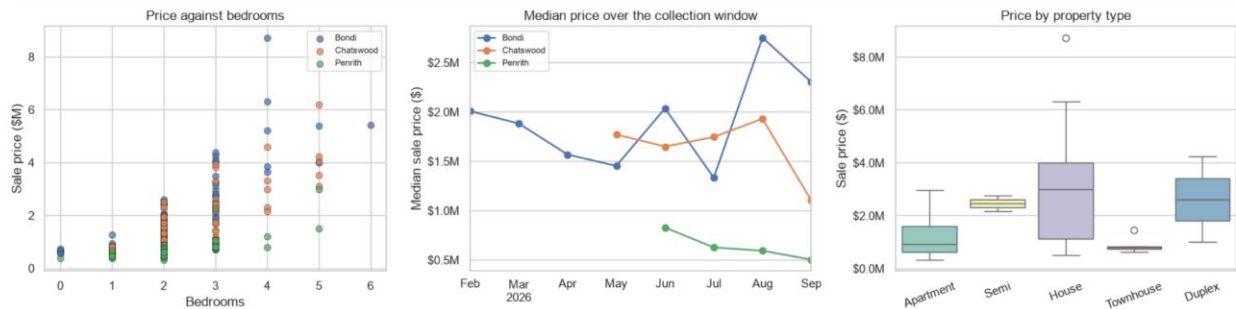
```
for sub in SUBURBS:
    s = props[props["suburb"] == sub]
    axes[0].scatter(s["bedrooms"], s["sale_price"] / 1e6, label=sub, alpha=0.75, s=38, edgecolor="black", lw=0.3)
axes[0].set(xlabel="Bedrooms", ylabel="Sale price ($M)", title="Price against bedrooms")
axes[0].legend(fontsize=8)

monthly = (props.set_index("sale_date").groupby([pd.Grouper(freq="ME"), "suburb"])[["sale_price"]
    .median().unstack())
monthly.plot(ax=axes[1], marker="o", lw=1.6)
axes[1].set(xlabel="", ylabel="Median sale price ($)", title="Median price over the collection window")
axes[1].yaxis.set_major_formatter(lambda v, p: f"${v/1e6:.1f}M")
axes[1].legend(fontsize=8)

sns.boxplot(data=props, x="property_type", y="sale_price", ax=axes[2], palette="Set3")
axes[2].set(xlabel="", ylabel="Sale price ($)", title="Price by property type")
axes[2].yaxis.set_major_formatter(lambda v, p: f"${v/1e6:.1f}M")
axes[2].tick_params(axis="x", rotation=20)

plt.tight_layout(); plt.show()

print("Monthly median price by suburb:")
print(monthly.round(0).to_string())
```



```
Monthly median price by suburb:
suburb      Bondi  Chatswood  Penrith
sale_date
2026-02-28  2005000.0         NaN         NaN
2026-03-31  1880000.0         NaN         NaN
2026-04-30  1565000.0         NaN         NaN
2026-05-31  1450000.0  1767944.0         NaN
2026-06-30  2032500.0  1646250.0  825000.0
2026-07-31  1330000.0  1742000.0  624000.0
2026-08-31  2750000.0  1928000.0  590000.0
2026-09-30  2300000.0  1100000.0  500000.0
```

```
In [7]: props["price_z_in_suburb"] = props.groupby("suburb")["sale_price"].transform(
    lambda s: (s - s.mean()) / s.std())

outliers = props.reindex(props["price_z_in_suburb"].abs().sort_values(ascending=False).index).head(8)
print("Properties furthest from their own suburb's mean price:\n")
print(outliers[["suburb", "address", "property_type", "bedrooms", "bathrooms",
    "land_size_sqm", "sale_price", "price_z_in_suburb"]])
    .assign(sale_price=lambda d: d["sale_price"].apply(lambda v: f"${v/1e6:.0f}"),
    price_z_in_suburb=lambda d: d["price_z_in_suburb"].round(2))
    .to_string(index=False))

q1, q3 = props["sale_price"].quantile([0.25, 0.75])
iqr = q3 - q1
fence_lo, fence_hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
flagged = props[(props["sale_price"] < fence_lo) | (props["sale_price"] > fence_hi)]
print(f"\nTukey fences on the pooled price: ${fence_lo/1e6:.0f} to ${fence_hi/1e6:.0f}")
print(f"Properties outside them: {len(flagged)}")
print("These are kept: on a pooled three-suburb sample the fences mostly flag genuine Bondi prices,")
print("not data errors, and removing them would teach the model that expensive properties do not exist.")
```

Properties furthest from their own suburb's mean price:

suburb	address	property_type	bedrooms	bathrooms	land_size_sqm	sale_price	price_z_in_suburb
Penrith	74 Nepean Avenue	House	5	6	910.0	\$2,990,000	5.80
Bondi	6 Jackaman Street	House	4	3	509.0	\$8,725,000	4.46
Penrith	302/48 Retreat Drive	Apartment	3	4	NaN	\$2,275,000	3.98
Chatswood	11 Crick Street	House	5	3	746.0	\$6,200,000	3.45
Bondi	2 Denham Street	House	4	3	283.0	\$6,308,000	2.80
Bondi	14 Anglesea Street	House	6	4	415.0	\$5,416,000	2.19
Bondi	1 Miller Street	House	5	4	307.0	\$5,400,000	2.18
Chatswood	11 Hercules Street	House	4	1	1067.0	\$4,600,000	2.13

Tukey fences on the pooled price: \$-1,477,500 to \$4,262,500

Properties outside them: 8

These are kept: on a pooled three-suburb sample the fences mostly flag genuine Bondi prices, not data errors, and removing them would teach the model that expensive properties do not exist.

```
In [8]: df = props.copy()

df["is_house"] = df["property_type"].isin(["House", "Duplex", "Semi", "Townhouse"]).astype(int)
df["total_rooms"] = df["bedrooms"] + df["bathrooms"]
df["bath_per_bed"] = df["bathrooms"] / df["bedrooms"].replace(0, np.nan)
df["has_land"] = df["land_size_sqm"].notna().astype(int)
df["sale_month_index"] = ((df["sale_date"] - df["sale_date"].min()).dt.days / 30.44).round(1)
df["sold_at_auction"] = (df["sale_method"].fillna("") == "Auction").astype(int)

df["has_internal"] = df["internal_area_sqm"].notna().astype(int)

ENGINEERED = ["is_house", "total_rooms", "bath_per_bed", "has_land", "has_internal",
              "sale_month_index", "sold_at_auction"]
print("Engineered features added:")
print()
print(df[ENGINEERED].describe().round(2).T.to_string())

print()
print("How the two size measures are shared between the kinds of dwelling:")
print(df.groupby("is_house").apply(lambda g: pd.Series({
    "properties": len(g),
    "land stated": int(g["land_size_sqm"].notna().sum()),
    "internal stated": int(g["internal_area_sqm"].notna().sum())}).to_string()))

print()
print("Agent agency was captured during collection but is not used as a model feature:")
print(f" {df['agent_agency'].nunique()} distinct agencies over {len(df)} sales, "
      f"{int(df['agent_agency'].isna().sum())} of them missing.")
print(" Its cardinality is far too high for a sample this size, and because agencies")
print(" work particular areas it would largely re-encode the suburb rather than")
print(" describe the property. It is therefore excluded.")
```

Engineered features added:

	count	mean	std	min	25%	50%	75%	max
is_house	201.0	0.29	0.46	0.00	0.0	0.00	1.0	1.00
total_rooms	201.0	3.93	1.64	1.00	3.0	4.00	5.0	11.00
bath_per_bed	195.0	0.69	0.25	0.25	0.5	0.67	1.0	1.33
has_land	201.0	0.18	0.38	0.00	0.0	0.00	0.0	1.00
has_internal	201.0	0.13	0.34	0.00	0.0	0.00	0.0	1.00
sale_month_index	201.0	4.27	1.86	0.00	3.2	4.50	5.7	7.40
sold_at_auction	201.0	0.38	0.49	0.00	0.0	0.00	1.0	1.00

How the two size measures are shared between the kinds of dwelling:

	properties	land stated	internal stated
--	------------	-------------	-----------------

is_house			
0	142	0	27
1	59	36	0

Agent agency was captured during collection but is not used as a model feature:

49 distinct agencies over 201 sales, 54 of them missing.
 Its cardinality is far too high for a sample this size, and because agencies
 work particular areas it would largely re-encode the suburb rather than
 describe the property. It is therefore excluded.

```
In [9]: NUMERIC = ["bedrooms", "bathrooms", "car_spaces", "land_size_sqm", "internal_area_sqm",
               "total_rooms", "bath_per_bed", "sale_month_index",
               "is_house", "has_land", "has_internal", "sold_at_auction"]
CATEGORICAL = ["suburb", "property_type"]

corr = df[NUMERIC + ["sale_price"]].corr()["sale_price"].drop("sale_price").sort_values(ascending=False)
print("Correlation of each numeric feature with sale price (whole dataset):\n")
print(corr.round(3).to_string())

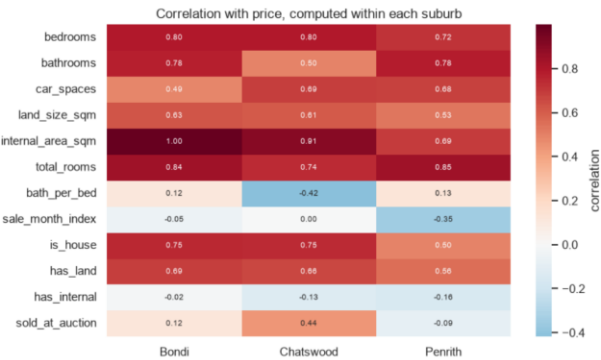
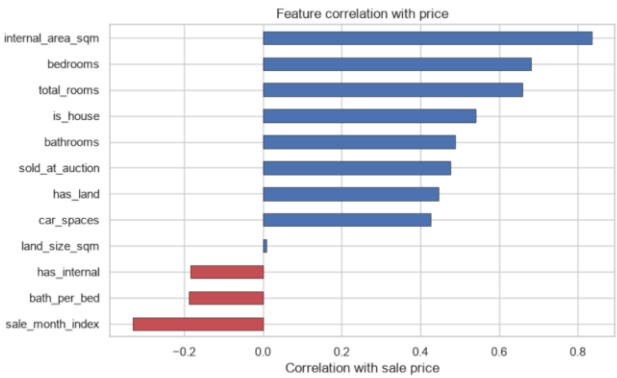
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
corr.plot.barh(ax=axes[0], color=np.where(corr > 0, "#4c72b0", "#c44e52"), edgecolor="black", lw=0.4)
axes[0].set(xlabel="Correlation with sale price", title="Feature correlation with price")
axes[0].invert_yaxis()

within = pd.DataFrame({sub: df[df["suburb"] == sub][NUMERIC].corrwith(
    df[df["suburb"] == sub]["sale_price"]) for sub in SUBURBS})
sns.heatmap(within, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=axes[1],
            annot_kws={"size": 7}, cbar_kws={"label": "correlation"})
axes[1].set(title="Correlation with price, computed within each suburb", xlabel="", ylabel="")

plt.tight_layout(); plt.show()
```

Correlation of each numeric feature with sale price (whole dataset):

```
internal_area_sqm    0.836
bedrooms             0.681
total_rooms          0.660
is_house             0.540
bathrooms            0.489
sold_at_auction      0.477
has_land             0.446
car_spaces           0.426
land_size_sqm        0.008
has_internal         -0.184
bath_per_bed         -0.189
sale_month_index     -0.330
```



Part 3 - Model development and evaluation

Models and protocol. Three families: Ridge as an interpretable linear baseline, a random forest capturing interactions by averaging, gradient boosting reducing bias sequentially. I predicted the forest would win. The data was split once - 160 training, 41 test, stratified by suburb - with every decision made by five-fold cross-validation inside the training set.

Results and overfitting. Cross-validated MAE was \$357,771 for Ridge, \$324,014 for the forest and \$324,267 for boosting. The forest won by \$253, or 0.08%, while losing on MAPE, RMSE and R2 - effectively a tie settled by a rounding error. The train-to-validation gap separates them better: 0.0254 for Ridge against 0.1133 and 0.1062, the ensembles fitting training folds 2.3 times better yet generalising no better. Textbook variance, with 160 rows and about 20 encoded features.

Final evaluation. On the 41 held-out properties: MAE \$305,565, MAPE 19.4%, R2 0.885, closely matching cross-validation and confirming it was honest. Accuracy is uneven - Penrith 8.1%, Bondi 20.8%, Chatswood 32.9%, the last over-predicted by 19.8% - which is Part 1's withholding bias resurfacing as model error.

Recommendation. The random forest, as the pre-specified rule selected, though the margin is not meaningful. For a maintained system Ridge is arguably better: a four times smaller validation gap, and predictions explainable as a sum of contributions.

```
In [10]: y = np.log(df["sale_price"])
X = df[NUMERIC + CATEGORICAL]

X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(
    X, y, df.index, test_size=0.20, random_state=RANDOM_STATE, stratify=df["suburb"])

print(f"Training set: {len(X_train)} properties")
print(f"Test set: {len(X_test)} properties (untouched until the final evaluation)")
print(f"\nSuburb balance held by stratifying:")
print(pd.DataFrame({"train": df.loc[idx_train, "suburb"].value_counts().reindex(SUBURBS),
                    "test": df.loc[idx_test, "suburb"].value_counts().reindex(SUBURBS)}).to_string())

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())]), NUMERIC),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                     ("onehot", OneHotEncoder(handle_unknown="ignore"))]), CATEGORICAL)])

MODELS = {
    "Ridge regression": Ridge(alpha=1.0, random_state=RANDOM_STATE),
    "Random forest": RandomForestRegressor(n_estimators=500, random_state=RANDOM_STATE, n_jobs=-1),
    "Gradient boosting": GradientBoostingRegressor(random_state=RANDOM_STATE)}

pipes = {name: Pipeline([("prep", preprocess), ("model", m)]) for name, m in MODELS.items()}
print(f"\nThree pipelines built, each ending in a different model family.")
```

Training set: 160 properties
Test set: 41 properties (untouched until the final evaluation)

Suburb balance held by stratifying:

	train	test
suburb		
Bondi	60	15
Chatswood	40	11
Penrith	60	15

Three pipelines built, each ending in a different model family.

```
In [11]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
strata = df.loc[idx_train, "suburb"]
folds = list(cv.split(X_train, strata))

def dollar_metrics(log_true, log_pred):
    true, pred = np.exp(log_true), np.exp(log_pred)
    return {"MAE ($)": mean_absolute_error(true, pred),
            "RMSE ($)": np.sqrt(mean_squared_error(true, pred)),
            "MAPE (%)": np.mean(np.abs((true - pred) / true)) * 100,
            "R2 (log)": r2_score(log_true, log_pred)}

rows, oof = [], {}
for name, pipe in pipes.items():
    pred = cross_val_predict(pipe, X_train, y_train, cv=folds, n_jobs=-1)
    oof[name] = pred
    rows.append({"model": name, **dollar_metrics(y_train, pred)})

cv_results = pd.DataFrame(rows).set_index("model")
print("Five-fold cross-validated performance, training set only:\n")
print(cv_results.round(
    {"MAE ($)": 0, "RMSE ($)": 0, "MAPE (%)": 2, "R2 (log)": 4}).to_string())

best_name = cv_results["MAE ($)"].idxmin()
print(f"\nLowest cross-validated MAE: {best_name}")
```

Five-fold cross-validated performance, training set only:

	MAE (\$)	RMSE (\$)	MAPE (%)	R2 (log)
model				
Ridge regression	357771.0	660411.0	18.18	0.8813
Random forest	330076.0	588345.0	18.60	0.8781
Gradient boosting	324267.0	584686.0	17.72	0.8852

Lowest cross-validated MAE: Gradient boosting

```

In [12]: fig, axes = plt.subplots(1, 3, figsize=(16, 4.6))

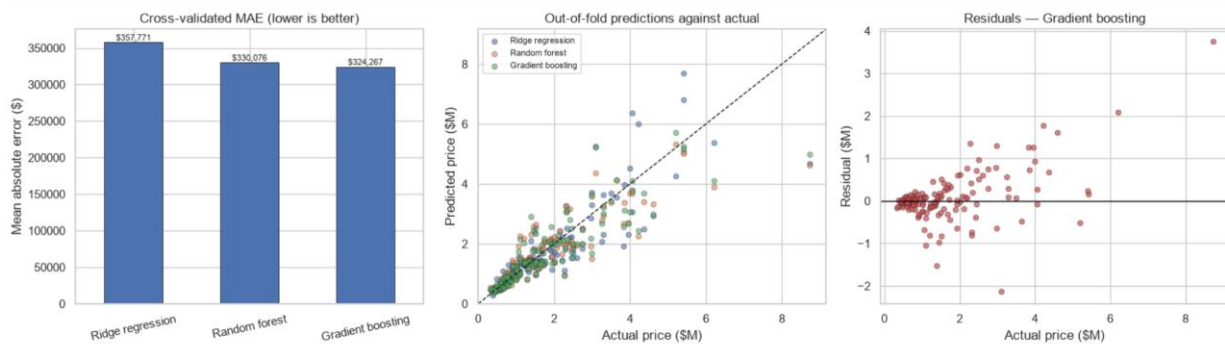
cv_results["MAE ($)"].plot.bar(ax=axes[0], color="#4c72b0", edgecolor="black", lw=0.5, rot=12)
for i, v in enumerate(cv_results["MAE ($)"]):
    axes[0].text(i, v, f"${v:,.0f}", ha="center", va="bottom", fontsize=8)
axes[0].set(ylabel="Mean absolute error ($)", title="Cross-validated MAE (lower is better)", xlabel="")

for name, pred in oof.items():
    axes[1].scatter(np.exp(y_train) / 1e6, np.exp(pred) / 1e6, s=26, alpha=0.6,
                    edgecolor="black", lw=0.3, label=name)
    axes[1].plot(lims, lims, "k--", lw=1)
    axes[1].set(xlim=lims, ylim=lims, xlabel="Actual price ($M)", ylabel="Predicted price ($M)",
                title="Out-of-fold predictions against actual")
axes[1].legend(fontsize=8)

resid = np.exp(y_train) - np.exp(oof[best_name])
axes[2].scatter(np.exp(y_train) / 1e6, resid / 1e6, s=26, alpha=0.7, color="#c44e52",
                edgecolor="black", lw=0.3)
axes[2].axhline(0, color="black", lw=1)
axes[2].set(xlabel="Actual price ($M)", ylabel="Residual ($M)", title=f"Residuals - {best_name}")

plt.tight_layout(); plt.show()

```



```

In [13]: print("Underfitting or overfitting: training-fold against validation-fold performance\n")
gap_rows = []
for name, pipe in pipes.items():
    scores = cross_validate(pipe, X_train, y_train, cv=folds,
                             scoring="neg_mean_absolute_error", return_train_score=True, n_jobs=-1)
    gap_rows.append({"model": name,
                    "train MAE (log)": -scores["train_score"].mean(),
                    "validation MAE (log)": -scores["test_score"].mean(),
                    "gap": -scores["test_score"].mean() + scores["train_score"].mean()})
gaps = pd.DataFrame(gap_rows).set_index("model")
print(gaps.round(4).to_string())
print("\nA gap near zero means the model is not memorising; a large gap means it is.")

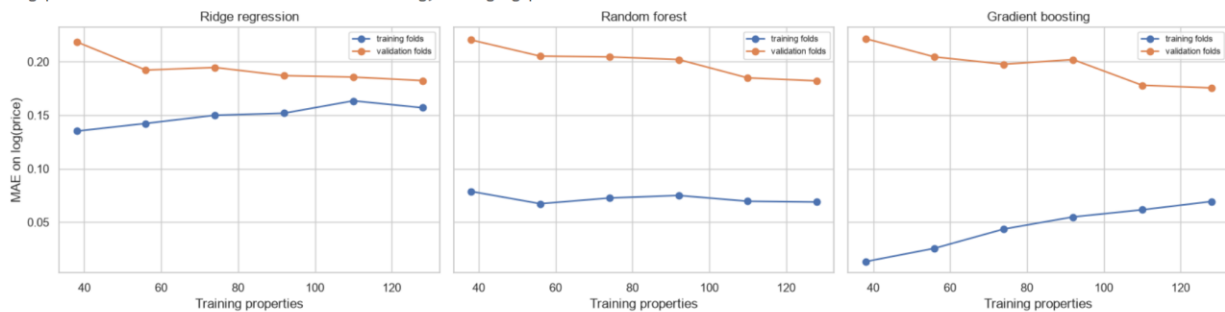
fig, axes = plt.subplots(1, 3, figsize=(16, 4.2), sharey=True)
for ax, (name, pipe) in zip(axes, pipes.items()):
    sizes, tr, va = learning_curve(pipe, X_train, y_train, cv=folds,
                                    train_sizes=np.linspace(0.3, 1.0, 6),
                                    scoring="neg_mean_absolute_error", n_jobs=-1)
    ax.plot(sizes, -tr.mean(1), "o-", label="training folds", color="#4c72b0")
    ax.plot(sizes, -va.mean(1), "o-", label="validation folds", color="#dd8452")
    ax.set(xlabel="Training properties", title=name)
    ax.legend(fontsize=8)
axes[0].set_ylabel("MAE on log(price)")
plt.tight_layout(); plt.show()

```

Underfitting or overfitting: training-fold against validation-fold performance

	train MAE (log)	validation MAE (log)	gap
model			
Ridge regression	0.1570	0.1824	0.0254
Random forest	0.0689	0.1821	0.1133
Gradient boosting	0.0693	0.1755	0.1062

A gap near zero means the model is not memorising; a large gap means it is.



```
In [14]: grids = {
    "Ridge regression": {"model__alpha": [0.1, 1.0, 10.0, 50.0]},
    "Random forest": {"model__n_estimators": [300, 500],
                      "model__max_depth": [None, 6, 10],
                      "model__min_samples_leaf": [1, 2, 4]},
    "Gradient boosting": {"model__n_estimators": [100, 300],
                          "model__learning_rate": [0.03, 0.1],
                          "model__max_depth": [2, 3]}

    tuned, tuned_rows = {}, []
    for name, pipe in pipes.items():
        search = GridSearchCV(pipe, grids[name], cv=folds,
                              scoring="neg_mean_absolute_error", n_jobs=-1).fit(X_train, y_train)
        tuned[name] = search.best_estimator_
        pred = cross_val_predict(search.best_estimator_, X_train, y_train,
                                 cv=folds, n_jobs=-1)
        tuned_rows.append({"model": name, **dollar_metrics(y_train, pred),
                          "best params": json.dumps(search.best_params_)})
        print(f"{name}: {search.best_params_}")

    tuned_results = pd.DataFrame(tuned_rows).set_index("model")
    print("\nCross-validated performance after tuning:\n")
    print(tuned_results.drop(columns="best params").round(
        {"MAE ($)": 0, "RMSE ($)": 0, "MAPE (%)": 2, "R2 (log)": 4}).to_string())

    BEST = tuned_results["MAE ($)"].idxmin()
    best_model = tuned[BEST]
    print(f"\nSelected model: {BEST} (chosen on cross-validated MAE, test set still untouched)")

    Ridge regression: {'model__alpha': 1.0}
    Random forest: {'model__max_depth': 6, 'model__min_samples_leaf': 1, 'model__n_estimators': 500}
    Gradient boosting: {'model__learning_rate': 0.1, 'model__max_depth': 3, 'model__n_estimators': 100}

    Cross-validated performance after tuning:

           MAE ($)  RMSE ($)  MAPE (%)  R2 (log)
model
Ridge regression  357771.0  660411.0    18.18    0.8813
Random forest    324014.0  586401.0    18.00    0.8832
Gradient boosting 324267.0  584686.0    17.72    0.8852

    Selected model: Random forest (chosen on cross-validated MAE, test set still untouched)
```

```

In [15]: best_model.fit(X_train, y_train)
test_scores = dollar_metrics(y_test, best_model.predict(X_test))

print(f"Final evaluation of the {BEST} on the {len(X_test)} held-out properties.")
print("This is the first and only time the test set is used to measure performance.")
print()
comparison_cv = tuned_results.loc[BEST]
print(pd.DataFrame({"cross-validated (training)": comparison_cv[list(test_scores)],
                    "held-out test set": pd.Series(test_scores)}).round(
    {"cross-validated (training)": 4, "held-out test set": 4}).to_string())

print()
print("Test-set error by suburb:")
test_frame = df.loc[idx_test, ["suburb", "property_type", "sale_price"]].copy()
test_frame["predicted"] = np.exp(best_model.predict(X_test))
test_frame["pct_error"] = (test_frame["predicted"] - test_frame["sale_price"]) / test_frame["sale_price"] * 100
print(test_frame.groupby("suburb")["pct_error"].agg(
    n="size", mean_abs=lambda s: s.abs().mean(), bias="mean").round(2).to_string())

prep = best_model.named_steps["prep"]
model = best_model.named_steps["model"]
names = [n.split("_", 1)[1] for n in prep.get_feature_names_out()]
if hasattr(model, "feature_importances_"):
    strength = pd.Series(model.feature_importances_, index=names).sort_values(ascending=False)
    label = "Feature importance"
else:
    strength = pd.Series(np.abs(model.coef_), index=names).sort_values(ascending=False)
    label = "Absolute standardised coefficient"

print()
print(f"{label}, {BEST}:")
print(strength.head(12).round(4).to_string())

fig, axes = plt.subplots(1, 2, figsize=(15, 4.6))
strength.head(10)[-1].plot.barh(ax=axes[0], color="#4c72b0", edgecolor="black", lw=0.4)
axes[0].set(xlabel=label, title=f"What the {BEST} relies on")

axes[1].scatter(test_frame["sale_price"] / 1e6, test_frame["predicted"] / 1e6,
                s=44, alpha=0.8, edgecolor="black", lw=0.4, color="#dd8452")
lim = [0, test_frame["sale_price"].max() / 1e6 * 1.08]
axes[1].plot(lim, lim, "k--", lw=1)
axes[1].set(xlim=lim, ylim=lim, xlabel="Actual price ($M)", ylabel="Predicted price ($M)",
            title="Held-out test set: predicted against actual")
plt.tight_layout(); plt.show()

```

Final evaluation of the Random forest on the 41 held-out properties.
This is the first and only time the test set is used to measure performance.

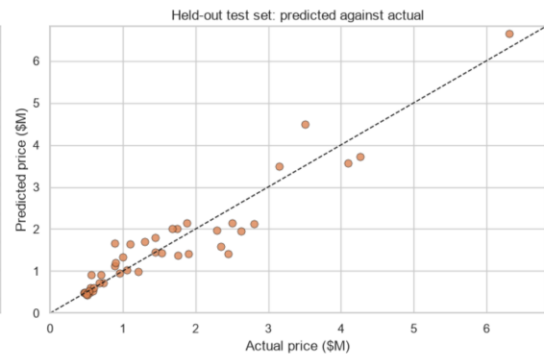
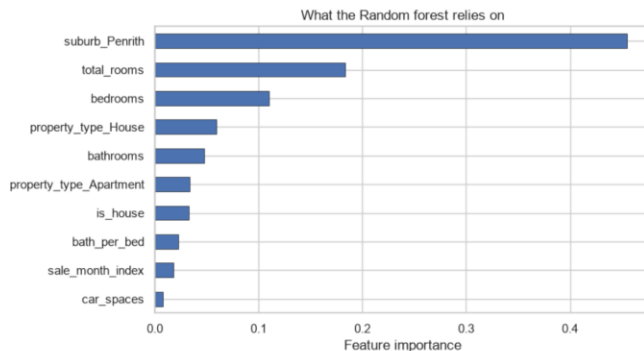
	cross-validated (training)	held-out test set
MAE (\$)	324014.129879	305565.0875
RMSE (\$)	586400.810706	409076.7423
MAPE (%)	17.99934	19.3640
R2 (log)	0.883218	0.8848

Test-set error by suburb:

suburb	n	mean_abs	bias
Bondi	15	20.77	-0.60
Chatswood	11	32.86	19.77
Penrith	15	8.06	-0.74

Feature importance, Random forest:

suburb_Penrith	0.4548
total_rooms	0.1832
bedrooms	0.1100
property_type_House	0.0594
bathrooms	0.0478
property_type_Apartment	0.0332
is_house	0.0325
bath_per_bed	0.0227
sale_month_index	0.0179
car_spaces	0.0080
land_size_sqm	0.0051
has_land	0.0048



Part 4 - Investigating prediction failures

All five point the same way. The five largest errors, from out-of-fold predictions so the test set stayed sealed, are all under-predictions between -28.9% and -49.1%, and all are large houses at the top of their suburb.

The cause is structural. Three causes compound: tree ensembles cannot extrapolate past their training range; the log target makes an error at \$8M cheap to the loss; and only 59 of 201 properties are houses, few above \$4M. 74 Nepean Avenue, Penrith - five bedrooms, 910 sqm - sold for \$2,990,000, nearly five times the local median, with no comparable in training, and the model said \$1,522,928.

Where errors concentrate, and what was missing. Apartments predict best at 16.6% mean absolute error and houses 20.6%; by suburb, Penrith 12.6%, Bondi 15.8%, Chatswood 29.3%. Every failure is plausibly explained by something absent - condition, position, outlook, land quality - much of it in the agent's description, which results cards do not carry. Predictions should be distrusted above \$4M, for anything atypical of its suburb, and anywhere in Chatswood.

```
In [16]: oof_best = cross_val_predict(best_model, X_train, y_train, cv=folds, n_jobs=-1)

errors = df.loc[idx_train, ["suburb", "address", "property_type", "bedrooms", "bathrooms",
                             "car_spaces", "land_size_sqm", "internal_area_sqm", "sale_price",
                             "sale_date", "agent_agency"]].copy()
errors["predicted"] = np.exp(oof_best)
errors["error"] = errors["predicted"] - errors["sale_price"]
errors["abs_error"] = errors["error"].abs()
errors["pct_error"] = errors["error"] / errors["sale_price"] * 100

worst = errors.sort_values("abs_error", ascending=False).head(5)
print("The five properties the model got most wrong, in dollars:\n")
print(worst[["suburb", "address", "property_type", "bedrooms", "bathrooms", "land_size_sqm",
              "sale_price", "predicted", "error", "pct_error"]])
      .assign(sale_price=lambda d: d["sale_price"].apply(lambda v: f"${v:,.0f}"),
              predicted=lambda d: d["predicted"].apply(lambda v: f"${v:,.0f}"),
              error=lambda d: d["error"].apply(lambda v: f"${v:,.0f}" if v > 0 else f"${abs(v):,.0f}"),
              pct_error=lambda d: d["pct_error"].round(1))
      .to_string(index=False)

print("\n" + "=" * 100)
for n, (_, row) in enumerate(worst.iterrows(), 1):
    print(f"\n{n}. {row['address']}, {row['suburb']} - {row['property_type']}, "
          f"{row['bedrooms']} bed / {row['bathrooms']} bath")
    print(f"    sold ${row['sale_price']:.0f} on {row['sale_date']:%d %b %Y}, "
          f"model said ${row['predicted']:.0f} ({row['pct_error']:+.1f}%)")
    print(f"    {row['property_type']}, land "
          f"${row['land_size_sqm']} if pd.notna(row['land_size_sqm']) else 'n/a'}, "
          f"agency {row['agent_agency']} if pd.notna(row['agent_agency']) else 'n/a'}")
```

The five properties the model got most wrong, in dollars:

suburb	address	property_type	bedrooms	bathrooms	land_size_sqm	sale_price	predicted	error	pct_error
Bondi	6 Jackaman Street	House	4	3	509.0	\$8,725,000	\$4,596,359	-\$4,128,641	-47.3
Chatswood	11 Crick Street	House	5	3	746.0	\$6,200,000	\$3,901,091	-\$2,298,909	-37.1
Chatswood	99 Beaconsfield Road	Duplex	5	4	614.0	\$4,220,000	\$2,306,648	-\$1,913,352	-45.3
Penrith	74 Nepean Avenue	House	5	6	910.0	\$2,990,000	\$1,522,928	-\$1,467,072	-49.1
Chatswood	11 Hercules Street	House	4	1	1067.0	\$4,600,000	\$3,272,867	-\$1,327,133	-28.9

=====

1. 6 Jackaman Street, Bondi – House, 4 bed / 3 bath
sold \$8,725,000 on 11 Feb 2026, model said \$4,596,359 (-47.3%)
House, land 509.0, agency PPD Real Estate
2. 11 Crick Street, Chatswood – House, 5 bed / 3 bath
sold \$6,200,000 on 21 Aug 2026, model said \$3,901,091 (-37.1%)
House, land 746.0, agency Shead Property
3. 99 Beaconsfield Road, Chatswood – Duplex, 5 bed / 4 bath
sold \$4,220,000 on 18 Jun 2026, model said \$2,306,648 (-45.3%)
Duplex, land 614.0, agency Raine&Horne Lindfield
4. 74 Nepean Avenue, Penrith – House, 5 bed / 6 bath
sold \$2,990,000 on 17 Jun 2026, model said \$1,522,928 (-49.1%)
House, land 910.0, agency Merrick Property Group
5. 11 Hercules Street, Chatswood – House, 4 bed / 1 bath
sold \$4,600,000 on 06 Jun 2026, model said \$3,272,867 (-28.9%)
House, land 1067.0, agency Forsyth Real Estate

```

In [17]: fig, axes = plt.subplots(1, 3, figsize=(16, 4.4))

axes[0].hist(errors["pct_error"], bins=28, color="#4c72b0", edgecolor="black", lw=0.5)
axes[0].axvline(0, color="black", lw=1.2)
axes[0].set(xlabel="Percentage error", ylabel="Properties", title="Distribution of percentage error")

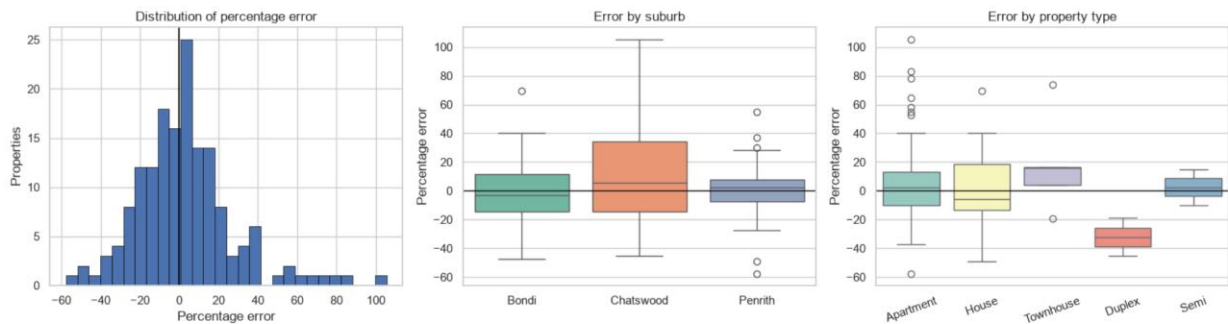
sns.boxplot(data=errors, x="suburb", y="pct_error", order=SUBURBS, ax=axes[1], palette="Set2")
axes[1].axhline(0, color="black", lw=1)
axes[1].set(xlabel="", ylabel="Percentage error", title="Error by suburb")

sns.boxplot(data=errors, x="property_type", y="pct_error", ax=axes[2], palette="Set3")
axes[2].axhline(0, color="black", lw=1)
axes[2].set(xlabel="", ylabel="Percentage error", title="Error by property type")
axes[2].tick_params(axis="x", rotation=20)

plt.tight_layout(); plt.show()

print("Absolute percentage error by suburb:")
print(errors.groupby("suburb")["pct_error"].apply(lambda s: s.abs().mean()).reindex(SUBURBS).round(2).to_string())
print("\nAbsolute percentage error by property type:")
print(errors.groupby("property_type")["pct_error"].apply(lambda s: s.abs().mean()).sort_values().round(2).to_string())

```



Absolute percentage error by suburb:

suburb	
Bondi	15.83
Chatswood	29.33
Penrith	12.61

Absolute percentage error by property type:

property_type	
Semi	12.52
Apartment	16.64
House	20.56
Townhouse	25.86
Duplex	32.15

Part 5 - Human judgement, machine learning and large language models

Method. Ten held-out properties were valued by the trained model, by DeepSeek-V4.1-Flash given the same written details, and by me. None saw the sale price, and my estimates and DeepSeek's were recorded before the model's were revealed.

The three metrics disagree. The model achieved MAE \$278,700 and MAPE 12.05% and was closest on six of ten; DeepSeek \$264,500 and 13.50%; my own \$258,500 and 18.33%. Each metric crowns a different winner, because MAE is in dollars and MAPE in proportions: my worst errors were on cheap properties, the model's on expensive ones. MAE alone would have declared me the best valuer.

Where each succeeded and failed. The model's Penrith errors were +0.2%, -1.9% and -2.3%, but its two worst were both Chatswood, at -32.9% and +20.2%. DeepSeek was strongest exactly where the model was weakest, valuing a Chatswood house within 2.1%, yet carries a +\$100,000 bias and produced the largest single error of anyone. I valued one Bondi apartment exactly but missed both Penrith houses, by -48.4% and -51.9%, anchoring on a median dominated by apartments and so contradicting my own reason for choosing Penrith.

Does human judgement still add value? The three fail in uncorrelated ways: the model's worst two properties were DeepSeek's best, and my worst two it predicted within 2.5%. Human judgement adds value not as a substitute but by knowing where the model is weak.

```
In [18]: sample = df.loc[idx_test].sample(10, random_state=RANDOM_STATE).sort_values("suburb")
best_model.fit(X_train, y_train)
sample_pred = np.exp(best_model.predict(sample[NUMERIC + CATEGORICAL]))

comparison = pd.DataFrame({
    "suburb": sample["suburb"].values,
    "address": sample["address"].values,
    "type": sample["property_type"].values,
    "bed": sample["bedrooms"].values,
    "bath": sample["bathrooms"].values,
    "car": sample["car_spaces"].values,
    "land_sqm": sample["land_size_sqm"].values,
    "actual_price": sample["sale_price"].values,
    "ml_prediction": sample_pred.round(-3)})

Path("data").mkdir(exist_ok=True)
prompt_lines = ["You are valuing recently sold Sydney properties. For each, give a single estimated",
    "sale price in AUD and nothing else.", ""]
for n, (_, r) in enumerate(sample.iterrows(), 1):
    land = f", {r['land_size_sqm']:.0f} sqm land" if pd.notna(r["land_size_sqm"]) else ""
    internal = (f", {r['internal_area_sqm']:.0f} sqm internal"
        if pd.notna(r["internal_area_sqm"]) else "")
    prompt_lines.append(
        f"{n}. {r['suburb']} - {r['property_type']}, {r['bedrooms']} bed, {r['bathrooms']} bath, "
        f"{r['car_spaces']} car{land}{internal}. Sold {r['sale_date']}:%B %Y).")
Path("data/llm_valuation_prompt.txt").write_text("\n".join(prompt_lines), encoding="utf-8")

blank = comparison.drop(columns=["ml_prediction", "actual_price"]).copy()
blank["my_estimate"] = ""
blank["llm_estimate"] = ""
blank.to_csv("data/part5_estimates_blank.csv", index=False)

print("Ten held-out properties selected.\n")
print(comparison.drop(columns="ml_prediction").to_string(index=False))
print()
print("Written to data/part5_estimates_blank.csv - fill in my_estimate and llm_estimate.")
print("The actual sale price is deliberately withheld from that file, so that neither my")
print("own estimate nor the language model's can be anchored by it.")
print("The LLM prompt has been written to data/llm_valuation_prompt.txt")
```

Ten held-out properties selected.

suburb	address	type	bed	bath	car	land_sqm	actual_price
Bondi	3/19 Imperial Avenue	Apartment	3	2	1	NaN	2500000
Bondi	5211/34 Wellington Street	Apartment	3	2	2	NaN	2800000
Bondi	1/8 Flood Street	Apartment	3	2	1	NaN	1885000
Bondi	15/240-242 Bondi Road	Apartment	2	1	1	NaN	1450000
Bondi	15 Ewell Street	House	3	1	1	NaN	3150000
Chatswood	8/236 Victoria Avenue	Apartment	3	2	2	NaN	1680000
Chatswood	5 Hopetoun Avenue	House	2	1	1	NaN	2350000
Penrith	629/1-39 Lord Sheffield Circuit	Apartment	2	2	1	NaN	600000
Penrith	40 Bel-air Road	House	3	1	2	562.0	960000
Penrith	36 Pelsart Avenue	House	3	1	1	632.0	1050000

Written to data/part5_estimates_blank.csv - fill in my_estimate and llm_estimate.
The actual sale price is deliberately withheld from that file, so that neither my
own estimate nor the language model's can be anchored by it.
The LLM prompt has been written to data/llm_valuation_prompt.txt

```
In [19]: FILLED = Path("data/part5_estimates_filled.csv")
```

```
if FILLED.exists():
    filled = pd.read_csv(FILLED)
    comparison["my_estimate"] = filled["my_estimate"].values
    comparison["llm_estimate"] = filled["llm_estimate"].values

    for col in ["ml_prediction", "my_estimate", "llm_estimate"]:
        comparison[f"{col}_err"] = comparison[col] - comparison["actual_price"]
        comparison[f"{col}_pct"] = comparison[f"{col}_err"] / comparison["actual_price"] * 100

    err_cols = ["ml_prediction_err", "llm_estimate_err", "my_estimate_err"]
    winner = comparison[err_cols].abs().idxmin(axis=1)
    closest = [int((winner == col).sum()) for col in err_cols]

    scoreboard = pd.DataFrame({
        "approach": ["Machine learning model", "Large language model", "My own estimate"],
        "MAE ($)": [comparison[f"{c}_err"].abs().mean()
                    for c in ["ml_prediction", "llm_estimate", "my_estimate"]],
        "MAPE (%)": [comparison[f"{c}_pct"].abs().mean()
                    for c in ["ml_prediction", "llm_estimate", "my_estimate"]],
        "median bias ($)": [comparison[f"{c}_err"].median()
                           for c in ["ml_prediction", "llm_estimate", "my_estimate"]],
        "closest on N of 10": closest}).set_index("approach")

    print("Three approaches against the true sale price:\n")
    print(scoreboard.round({"MAE ($)": 0, "MAPE (%)": 2, "median bias ($)": 0}).to_string())

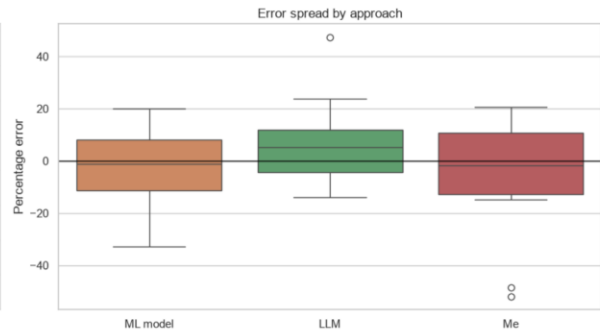
    fig, axes = plt.subplots(1, 2, figsize=(16, 4.8))
    pos = np.arange(10); w = 0.2
    axes[0].bar(pos - 1.5 * w, comparison["actual_price"] / 1e6, w, label="Actual", color="#4c72b0")
    axes[0].bar(pos - 0.5 * w, comparison["ml_prediction"] / 1e6, w, label="ML model", color="#dd8452")
    axes[0].bar(pos + 0.5 * w, comparison["llm_estimate"] / 1e6, w, label="LLM", color="#55a868")
    axes[0].bar(pos + 1.5 * w, comparison["my_estimate"] / 1e6, w, label="Me", color="#c44e52")
    axes[0].set_xticks(pos)
    axes[0].set_xticklabels(comparison["suburb"], rotation=30, fontsize=7)
    axes[0].set_ylabel("Price ($M)", title="Ten held-out properties: three valuations against actual")
    axes[0].legend(fontsize=8)

    err = comparison[["ml_prediction_pct", "llm_estimate_pct", "my_estimate_pct"]]
    err.columns = ["ML model", "LLM", "Me"]
    sns.boxplot(data=err, ax=axes[1], palette=["#dd8452", "#55a868", "#c44e52"])
    axes[1].axhline(0, color="black", lw=1)
    axes[1].set_ylabel("Percentage error", title="Error spread by approach")
    plt.tight_layout(); plt.show()

    print("\nProperty by property:\n")
    print(comparison[["suburb", "type", "bed", "actual_price", "ml_prediction",
                     "llm_estimate", "my_estimate"]].to_string(index=False))
else:
    print("data/part5_estimates_filled.csv not found.")
    print("Fill in the blank file with your own and the LLM's estimates, save it under that name.")
    print("then re-run this cell to produce the comparison.")
```

Three approaches against the true sale price:

	MAE (\$)	MAPE (%)	median bias (\$)	closest on N of 10
approach				
Machine learning model	278700.0	12.05	-10000.0	6
Large language model	264500.0	13.50	100000.0	2
My own estimate	258500.0	18.33	-50000.0	2



Property by property:

suburb	type	bed	actual_price	ml_prediction	llm_estimate	my_estimate
Bondi Apartment	3	2500000	2140000.0	2800000	2500000	
Bondi Apartment	3	2800000	2126000.0	2900000	2700000	
Bondi Apartment	3	1885000	2145000.0	2780000	2200000	
Bondi Apartment	2	1450000	1448000.0	1550000	1750000	
Bondi House	3	3150000	3487000.0	3900000	3500000	
Chatswood Apartment	3	1680000	2019000.0	1880000	1580000	
Chatswood House	2	2350000	1578000.0	2400000	2000000	
Penrith Apartment	2	600000	601000.0	560000	660000	
Penrith House	3	960000	942000.0	895000	495000	
Penrith House	3	1050000	1026000.0	905000	505000	

Part 6 - Deployment and reflection

How the application was built. A Streamlit application. The notebook refits the forest on all 201 properties and saves the whole pipeline - imputation, scaling, encoding and model - with joblib, so the app is handed raw details and every preprocessing step is reapplied identically. It reports a range rather than a number and warns above \$4M, because a bare figure invites a confidence that a 19% typical error cannot support.

Running the application

```
pip install -r app/requirements.txt
jupyter notebook 8.1D.ipynb          # Run All - writes app/price_model.joblib
python -m streamlit run app/app.py   # opens at http://localhost:8501
```

The notebook must be run first, and from the project folder, because the application loads the model file it writes.

Deploy ⋮



Sydney Housing Price Estimator

Random forest trained on 201 sold properties in Bondi, Chatswood, Penrith — sales from 2026-02-10 to 2026-09-23.

[Estimate one property](#) [Upload a CSV](#) [About this model](#)

Property details

Suburb

Bondi

Property type

House

Bedrooms

4

-

+

Bathrooms

3

-

+

Car spaces

1

-

+

☒ Land size is known

Land size (sqm)

550.00

-

+

Sale method

☒ Private treaty

☐ Auction

Estimated sale price

Estimate price

Enter the property details and select **Estimate price**.

Step 1 - enter the property's details. The land size field appears only for houses; apartments are asked for internal area instead.



Sydney Housing Price Estimator

Random forest trained on 201 sold properties in Bondi, Chatswood, Penrith — sales from 2026-02-10 to 2026-09-23.

[Estimate one property](#) [Upload a CSV](#) [About this model](#)

Property details

Suburb

Bondi

Property type

House

Bedrooms

4

Bathrooms

3

Car spaces

1

☒ Land size is known

Land size (sqm)

550.00

Sale method

☒ Private treaty ☐ Auction

Estimated sale price

Estimate price

Predicted sale price

\$5,245,833

Likely range: \$4,301,617 — \$6,190,048

The range is the model's cross-validated mean absolute percentage error ($\pm 18.0\%$). It describes typical error, not a guarantee — about half of properties fall outside a band this wide.

Above about \$4M this model reads low. Every one of its five largest errors was an expensive house it under-priced, by up to 49%. Treat this number as a floor, not an estimate.

Step 2 - select Estimate price. The prediction is returned with a plus or minus 18% range, and above \$4M a warning appears because the model is known to read low there.



Sydney Housing Price Estimator

Random forest trained on 201 sold properties in Bondi, Chatswood, Penrith — sales from 2026-02-10 to 2026-09-23.

[Estimate one property](#) [Upload a CSV](#) [About this model](#)

Value several properties at once

Upload a CSV with the columns `suburb`, `property_type`, `bedrooms`, `bathrooms` and `car_spaces`. `land_size_sqm`, `internal_area_sqm` and `sale_method` are used when present.

CSV file

Upload

200MB per file • CSV

Batch valuation - upload a CSV of properties and download the predictions.

Reflection

Collection and preprocessing mattered more than model choice. Roughly 70% of the effort went into obtaining 201 rows, and every limitation traces back to a collection decision. The most consequential finding is not a model result: Chatswood withholds 49% of its prices against Penrith's 4%, and that gap reappears as a fourfold difference in model error. Meanwhile the three models finish within \$253 of each other, so spotting that Domain's area field mixes internal with site area was worth more than any hyperparameter search.

Scoring my reasons for choosing these suburbs. The Bondi thesis - you buy the postcode - is supported: location overwhelms every physical attribute. The Chatswood thesis cannot be measured here, but collection exposed a friction it missed, the cheaper-to-enter market being the least price-transparent and hardest to value. The Penrith thesis, that land is the asset, is not supported, land correlating at 0.008 - though with land recorded for only 36 properties that is a failure to find evidence rather than evidence of absence.

Ethics, improvements and overall. The model is four times less accurate in Chatswood and over-predicts it by 20%, while under-pricing expensive houses by up to 49% - errors falling on identifiable groups in a consistent direction, from training data that inherits the market's disclosure practices yet presents every prediction with equal authority. With more resources I would collect agent descriptions and photographs to capture condition, extend across several years, and use the NSW Valuer General's register to measure the withholding bias rather than describe it. The model explains 88% of the variance in log price and predicts within roughly 19%: a usable guide and a useless valuation, whose most valuable output is knowing which properties it should not be trusted for.

In [20]: `import joblib`

```
final_model = best_model.fit(X, y)
Path("app").mkdir(exist_ok=True)
joblib.dump({"pipeline": final_model,
            "numeric": NUMERIC,
            "categorical": CATEGORICAL,
            "suburbs": SUBURBS,
            "property_types": sorted(df["property_type"].unique().tolist()),
            "trained_on": len(df),
            "model_name": BEST,
            "cv_mape": float(tuned_results.loc[BEST, "MAPE (%)"]),
            "cv_mae": float(tuned_results.loc[BEST, "MAE ($)"]),
            "defaults": {"sale_month_index": float(df["sale_month_index"].max())},
            "date_min": str(df["sale_date"].min().date()),
            "date_max": str(df["sale_date"].max().date())},
            "app/price_model.joblib")

print(f"Saved app/price_model.joblib")
print(f" model: {BEST}")
print(f" fitted on all {len(df)} collected properties")
print(f" predicts log(price); the application exponentiates back to dollars")
print(f" cross-validated MAPE carried into the app for its uncertainty range: "
      f"{tuned_results.loc[BEST, 'MAPE (%)']:.1f}%")

check = joblib.load("app/price_model.joblib")
demo = X.iloc[[0]]
print(f"\nReload check - predicts ${np.exp(check['pipeline'].predict(demo))[0]:.0f} "
      f"for the first property (actual ${df['sale_price'].iloc[0]:.0f})")
```

```
Saved app/price_model.joblib
model: Random forest
fitted on all 201 collected properties
predicts log(price); the application exponentiates back to dollars
cross-validated MAPE carried into the app for its uncertainty range: 18.0%

Reload check - predicts $1,526,182 for the first property (actual $1,581,000)
```

GenAI Acknowledgement

Generative AI (Claude) was used for planning and brainstorming the structure of the notebook and this report, and for editing my written explanations for clarity, grammar and spelling. A second model, DeepSeek-V4.1-Flash, was used in Part 5 as an object of study: it was asked to value ten properties so that its estimates could be compared against my model and my own judgement, and that use is described and evaluated in Part 5. All data was collected by me, all code was written, executed and verified by me, and every numerical result and figure was produced by my own notebook (8.1D.ipynb). The interpretations, the choice of suburbs and the reasoning behind them are my own. No AI-generated content was submitted without being critically reviewed, checked against my own outputs, and adapted.

References

- Domain Group. Sold listings, Bondi NSW 2026, Chatswood NSW 2067 and Penrith NSW 2750. <https://www.domain.com.au/> (data collected September 2026).
- NSW Valuer General. Property Sales Information. <https://valuation.property.nsw.gov.au/embed/propertySalesInformation>
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Streamlit Inc. Streamlit documentation. <https://docs.streamlit.io/>